

```

1 import components.naturalnumber.NaturalNumber;
2 import components.naturalnumber.NaturalNumber2;
3 import components.simplewriter.SimpleWriter;
4 import components.simplewriter.SimpleWriter1L;
5
6 /**
7  * Program with implementation of {@code NaturalNumber} secondary operation
8  * {@code root} implemented as static method.
9  *
10 * @author Eric Zhou
11 *
12 */
13 public final class NaturalNumberRoot {
14
15     /**
16      * Private constructor so this utility class cannot be instantiated.
17      */
18     private NaturalNumberRoot() {
19     }
20
21     /**
22      * Updates {@code n} to the {@code r}-th root of its incoming value.
23      *
24      * @param n
25      *         the number whose root to compute
26      * @param r
27      *         root
28      * @updates n
29      * @requires r >= 2
30      * @ensures n ^ (r) <= #n < (n + 1) ^ (r)
31      */
32     public static void root(NaturalNumber n, int r) {
33         assert n != null : "Violation of: n is not null";
34         assert r >= 2 : "Violation of: r >= 2";
35
36         NaturalNumber lowEnough = new NaturalNumber2(0);
37         NaturalNumber tooHigh = new NaturalNumber2(n);
38
39         while (tooHigh.compareTo(lowEnough) > 0) {
40             tooHigh.increment(); //change the bound to n+1
41             NaturalNumber guess = new NaturalNumber2(lowEnough);
42             guess.add(tooHigh);
43             guess.divide(new NaturalNumber2(2)); //get the middle
44             NaturalNumber power = new NaturalNumber2(guess);
45             power.power(r);
46
47             if (power.compareTo(n) <= 0) { //judge the power
48                 lowEnough.copyFrom(guess);
49             } else {
50                 tooHigh.copyFrom(guess);
51             }
52             tooHigh.decrement();
53         }
54         n.copyFrom(lowEnough);
55     }
56
57     /**
58      * Main method.
59      *
60      * @param args
61      *         the command line arguments
62      */
63     public static void main(String[] args) {
64         SimpleWriter out = new SimpleWriter1L();
65

```

```

66    final String[] numbers = { "0", "1", "13", "1024", "189943527", "0", "1", "13",
67        "4096", "189943527", "0", "1", "13", "1024", "189943527", "82", "82",
68        "82", "82", "82", "9", "27", "81", "243", "143489073", "2147483647",
69        "2147483648", "9223372036854775807", "9223372036854775808",
70        "618970019642690137449562111", "162259276829213363391578010288127",
71        "170141183460469231731687303715884105727" };
72    final int[] roots = { 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 15, 15, 15, 15, 15, 2, 3, 4,
73        5, 15, 2, 3, 4, 5, 15, 2, 2, 3, 3, 4, 5, 6 };
74    final String[] results = { "0", "1", "3", "32", "13782", "0", "1", "2", "16",
75        "574", "0", "1", "1", "1", "3", "9", "4", "3", "2", "1", "3", "3", "3",
76        "3", "3", "46340", "46340", "2097151", "2097152", "4987896", "2767208",
77        "2353973" };
78
79    for (int i = 0; i < numbers.length; i++) {
80        NaturalNumber n = new NaturalNumber2(numbers[i]);
81        NaturalNumber r = new NaturalNumber2(results[i]);
82        root(n, roots[i]);
83        if (n.equals(r)) {
84            out.println("Test " + (i + 1) + " passed: root(" + numbers[i] + ", "
85                + roots[i] + ") = " + results[i]);
86        } else {
87            out.println("*** Test " + (i + 1) + " failed: root(" + numbers[i] + ", "
88                + roots[i] + ") expected <" + results[i] + "> but was <" + n
89                + ">");
90        }
91    }
92
93    out.close();
94 }
95
96 }
97

```